Application Module: Design Document

Large-Scale Panel Data Analysis via Online TWFE

Hwang & Lee (2026) — *Online Updating for Linear Panel Regressions*

March 2026

Contents

1 Overview and Motivation **1**

2 Model and Estimands **1**

3 Prerequisites and Design Constraints **2**

3.1 Unit-Boundary Constraint 2

3.2 Sorted Input Requirement 2

3.3 Pre-specified Calendar Time Support 2

4 Four-Phase Workflow **3**

4.1 Phase 0 — Data Inspection 3

4.2 Phase 1 — Preprocessing: Sort 3

4.3 Phase 2 — Chunked State Accumulation 3

4.4 Phase 3 — Finalisation 5

5 Correctness Guarantees **5**

6 Comparison with Batch Methods **5**

7 Implementation Plan **6**

7.1 Files to be Created or Extended 6

7.2 Streaming DGP Design (`make_panel_52_stream`) 6

7.3 Application Script Structure (`streaming_bigdata.R`) 6

8 Known Limitations and Scope **7**

1 Overview and Motivation

The simulation study (Section 5.2) demonstrates that the online TWFE algorithm achieves numerical equivalence with a full-sample batch estimator while processing units sequentially. In that setting, data are *generated* unit by unit, so the full panel is never materialised in memory.

The application module addresses a complementary and practically important scenario: a *pre-existing* large dataset—stored as a flat file or in a database—that is too large to load into RAM at once. The canonical example is a dataset with $N = 10^8$ individuals and $T = 10$ time periods, whose raw size may easily exceed 10 GB.

Standard batch estimators such as `plm` require the entire `pdata.frame` to reside in memory before estimation begins. The benchmark results already show that `plm` exhausts a 16 GB memory limit at $N \approx 2 \times 10^7$. The online TWFE algorithm, by contrast, maintains a fixed-size state object of $O(p^2)$ scalars regardless of N, and processes the data one chunk at a time.

Remark 1. *The distinction between the simulation and application settings lies entirely in* how data enter the algorithm*, not in the algorithm itself. In both cases the same algebraically exact update equations (Propositions 1–3) are applied; only the data ingestion pipeline differs.*

2 Model and Estimands

The target model is the two-way fixed-effects (TWFE) panel regression

$$Y_{it} \;=\; X_{it}'\beta + \alpha_i + \lambda_t + u_{it}, \qquad i = 1, \ldots, N, \quad t \in \mathcal{S}_i \subseteq \{1, \ldots, T\},$$

where α_i denotes individual fixed effects and λ_t calendar-time fixed effects. The panel is *unbalanced*: each unit i is observed on a subset $\mathcal{S}_i$ of the T calendar periods.

The three estimands of interest are:

(i) the slope coefficient vector $\hat{\beta}$ (via Algorithm 1);

(ii) the classical variance $\widehat{V}_0 = \hat{\sigma}^2(\dot{Z}'\dot{Z})^{-1}$; and

(iii) the cluster-robust (Arellano HC0) variance $\hat{V}_{cr}$.

All three are produced in a *single pass* over the data chunks.

3 Prerequisites and Design Constraints

3.1 Unit-Boundary Constraint

The within transformation for individual i,

$$\dot{Z}_i = Z_i - \bar{Z}_i, \qquad \dot{Y}_i = Y_i - \bar{Y}_i,$$

requires *all* observations of unit i to be available simultaneously. Consequently, a fundamental constraint on any chunked design is:

Design Principle 1 (Unit atomicity)**.** *No chunk may split the observations of a single unit across two chunks. Every unit's rows must appear entirely within one chunk.*

This implies that chunks are defined in terms of *number of units*, not number of rows. For an unbalanced panel, the row count of a chunk of C units is $\sum_{i \in \text{chunk}} T_i$, which varies across chunks.

3.2 Sorted Input Requirement

Unit atomicity can only be guaranteed if the dataset is sorted by the individual identifier (`id`) prior to chunked reading.

Design Principle 2 (Sorted input)**.** *The dataset must be sorted by* `id` *in non-decreasing order before the chunked analysis begins.*

A dataset sorted by calendar time, for instance, would interleave the rows of different units and make unit-boundary detection impossible without a full pass.

3.3 Pre-specified Calendar Time Support

Let $\mathcal{T} = \{t_1, t_2, \ldots, t_T\}$ denote the full set of calendar times present in the dataset. The $p = k + T - 1$ dimensional parameter space is fixed at initialisation. If $\mathcal{T}$ were allowed to grow during chunked processing, Algorithm 3 (new calendar time) would be triggered, expanding the parameter dimension and requiring re-projection of previously accumulated sufficient statistics onto the enlarged parameter space.

Design Principle 3 (Pre-specified T_{support})**.** *The full calendar time support T is determined before processing begins and passed to the initialisation step. All chunks are assumed to contain only time values within $\{1, \ldots, T\}$.*

In practice, T is either known from the study design (e.g. a 10-year panel has $T = 10$) or is determined with a single lightweight scan of only the `time` column of the file, which is fast even for very large files.

4 Four-Phase Workflow

4.1 Phase 0 — Data Inspection

Before any estimation, the analyst performs a lightweight scan to collect structural metadata:

- Column names and data types (`id`, `time`, `y`, $x_1, \ldots, x_k$).
- The set of unique calendar times $\{t_1, \ldots, t_T\}$, from which T_{support} and `baseline_time` are determined.
- Whether the file is already sorted by `id`.

This phase reads only the `id` and `time` columns (a small fraction of the total file size) and produces no large intermediate objects.

4.2 Phase 1 — Preprocessing: Sort

If the file is not already sorted by `id`, an external sort is required. For a 10 GB file this is infeasible with R's in-memory sort; instead, one of the following approaches is used:

- Unix `sort` utility (fastest; operates directly on disk).
- Apache Arrow (`open_dataset` → `arrange` → `write_csv_arrow`).

- A database `ORDER BY id` query.

Phase 1 is a *one-time cost*. If the data source is a database or a file system that supports querying by id range, this phase may be bypassed.

4.3 Phase 2 — Chunked State Accumulation

This is the core of the application module. Phase 2 implements the pipeline:

$$\texttt{otwfe_init} \;\rightarrow\; \underbrace{\texttt{otwfe_update}(\text{chunk } 1) \;\rightarrow\; \cdots \;\rightarrow\; \texttt{otwfe_update}(\text{chunk } C)}_{\text{repeat for each chunk}}.$$

Chunk boundary detection. Given the sorted file, the unit boundaries are identified by run-length encoding of the `id` column. This produces, for each unit, the number of rows it occupies. Cumulative sums of these lengths yield exact row offsets for every unit, allowing each chunk to be read as a contiguous block of exactly C units. This scan is performed once and the boundary table (a vector of N integers) is retained in memory; for $N = 10^8$, this amounts to approximately 400 MB—a fixed cost independent of the file size.

Initialisation (`otwfe_init`). The handle is created with:

- the covariate column names $x_1, \ldots, x_k$;
- T_{support} (from Phase 0);
- `baseline_time` (the omitted time dummy); and
- a verbosity flag.

At this point no data have been read. The state object $\mathcal{S}$ is uninitialised; the handle is an empty shell.

First chunk (`otwfe_update`, call 1). Processing the first chunk involves two sub-steps:

1. **Warm-up initialisation.** A small subset of units (at least $3p$ units, minimum 30) is selected from the first chunk by a greedy cover algorithm that guarantees every calendar time $t \in \{1, \ldots, T\}$ is represented at least once in the warm-up set. These units are used to compute an initial OLS estimate via the full within-regression, producing the initial state
$$\mathcal{S}_0 = \big\{(\dot{Z}'\dot{Z})^{-1},\ \hat{\beta},\ \hat{\sigma}^2,\ N_{\text{warm}},\ n_{\text{warm}},\ M_{ss},\ A_N,\ B_N\big\}.$$
2. **Remaining units.** All units in the first chunk beyond the warm-up set are processed by the Rcpp batch update (`alg1_batch_cpp`), which adds their sufficient statistics to $\mathcal{S}$ in a single pass.

Subsequent chunks (`otwfe_update`, calls 2 onward). Each subsequent chunk contains only new units not yet seen by the algorithm. The Rcpp batch routine processes the entire chunk in a single pass:

1. For each unit i in the chunk, compute $S_i = \dot{Z}_i'\dot{Z}_i,\ \ s_i = \dot{Z}_i'\dot{Y}_i,\ \ \text{SSy}_i = \dot{Y}_i'\dot{Y}_i$, and $\text{vec}(S_i)$.

2. Accumulate $\sum S_i$, $\sum s_i$, $\sum \mathrm{SSy}_i$ into running totals.

3. Accumulate the three Vcr building blocks:

$$M_{ss} \mathrel{+}= s_i s_i', \qquad A_N \mathrel{+}= s_i \,\mathrm{vec}(S_i)', \qquad B_N \mathrel{+}= \mathrm{vec}(S_i)\,\mathrm{vec}(S_i)'.$$

4. Update $(\dot{Z}'\dot{Z})^{-1}$, $\hat{\beta}$, and $\hat{\sigma}^2$ using the rank-update formula.

The key property that makes this chunk-safe is:

Remark 2 (Chunk-safe Vcr accumulation)**.** *The matrices M_{ss}, A_N, B_N are defined as sums over individual units. Because addition is commutative and associative, these sums can be split across any number of chunks and accumulated in any order. The final Vcr is computed* after all chunks have been processed*, using the final $\hat{\beta}$:*

$$M \;=\; M_{ss} - A_N\big(\hat{\beta}' \otimes I_p\big) - \big(\hat{\beta}' \otimes I_p\big)' A_N' + \big(\hat{\beta}' \otimes I_p\big) B_N \big(\hat{\beta}' \otimes I_p\big)',$$

$$\hat{V}_{cr} \;=\; (\dot{Z}'\dot{Z})^{-1}\, M\, (\dot{Z}'\dot{Z})^{-1}.$$

This formula is algebraically exact regardless of how many chunks were used or in what order units were processed.

Memory profile. After each call to `otwfe_update`, the chunk data frame is immediately deleted from memory. The only object persisting between chunks is the state handle $\mathcal{S}$, whose size is:

$$\underbrace{p \times p}_{\text{inv}} + \underbrace{p}_{\hat{\beta}} + \underbrace{1}_{\hat{\sigma}^2} + \underbrace{p \times p}_{M_{ss}} + \underbrace{p \times p^2}_{A_N} + \underbrace{p^2 \times p^2}_{B_N} \;=\; O(p^4) \text{ scalars.}$$

For $p = 4$ ($k = 2$, $T = 3$), this is fewer than 200 double-precision numbers — well under one kilobyte. This footprint is *independent of N*.

4.4 Phase 3 — Finalisation

After all chunks have been processed, `otwfe_finalize` performs:

1. Compute $\hat{V}_{cr}$ from the accumulated M_{ss}, A_N, B_N, and the final $\hat{\beta}$ using the formula in Remark 2.

2. Package $\hat{\beta}$, $\hat{V}_0$, $\hat{V}_{cr}$, and metadata into the standard `otwfe` return object.

This step is $O(p^4)$ and takes negligible time.

5 Correctness Guarantees

The chunked pipeline is *not* an approximation. The following equivalences hold exactly (up to floating-point rounding at double precision):

(i) $\hat{\beta}_{\text{chunk}} = \hat{\beta}_{\text{batch}}$: The coefficient vector produced by processing C-unit chunks is identical to the OLS estimate from a single batch regression on the full dataset.

(ii) $\hat{V}_{0,\text{chunk}} = \hat{V}_{0,\text{batch}}$: The classical variance $\hat{\sigma}^2(\dot{Z}'\dot{Z})^{-1}$ is exact because both $\hat{\sigma}^2$ and $(\dot{Z}'\dot{Z})^{-1}$ accumulate exactly across chunks.

(iii) $\hat{V}_{cr,\text{chunk}} = \hat{V}_{cr,\text{batch}}$: By Remark 2, the single-pass Vcr formula evaluated at the final $\hat{\beta}$ yields the same result as the Arellano HC0 sandwich estimator applied to the full sample.

These equivalences were verified numerically in the simulation study at $N = 10^6$, 5×10^6, and 10^7, with maximum coefficient discrepancies on the order of 10^{-13}, consistent with double-precision arithmetic.

6 Comparison with Batch Methods

Table 1: Online TWFE (chunk API) vs. `plm` (batch)

Property	Online TWFE (chunk)	`plm`
Full dataset in RAM required	No	Yes
RAM usage scales with N	No ($O(p^4)$)	Yes ($O(nk)$)
Feasible at $N = 2 \times 10^7$	Yes	No (OOM on 16 GB)
Algebraically exact	Yes	Yes
Cluster-robust SE (Arellano HC0)	Yes (single pass)	Yes (post-estimation)
Requires sorted input	Yes	No
One-time preprocessing cost	Yes (sort)	None

The primary trade-off is that the chunk API requires the dataset to be sorted by `id` in advance. For a one-time analysis of a persistent dataset, this is a mild requirement. For a truly streaming environment where data are generated in unit order (as in the simulation study), sorting is not needed.

7 Implementation Plan

7.1 Files to be Created or Extended

`Simulation/sim_dgp.R` **(extension):** A new function `make_panel_52_stream()` will be added to the existing DGP file. This function returns a *closure-based iterator* that, on each call to `next_chunk()`, generates and returns exactly one chunk of C units without retaining any state beyond the chunk index, an id offset counter, and the pre-generated λ_t vector (T scalars).

`Application/streaming_bigdata.R`**:** Main demonstration script that exercises the four-phase workflow on datasets of increasing size, reports timing and accuracy, and produces the summary table for the paper.

7.2 Streaming DGP Design (`make_panel_52_stream`)

The function is designed with the following properties:

- **Minimal pre-allocation.** At construction, only λ_t (T scalars) is drawn. Nothing proportional to N is allocated.
- **Per-chunk generation.** Each call to `next_chunk()` draws α_i, T_i, calendar time subsets, X_{it}, u_{it}, and Y_{it} for the next C units, constructs the data frame, and returns it. The generator's internal state advances by C units.

- **Reproducibility.** Each chunk uses a seed derived from the global seed and the chunk index. This ensures that any particular chunk produces the same data regardless of how many chunks have been processed before it.
- **Globally unique IDs.** Chunk c produces unit IDs from $(c-1)C+1$ to $\min(cC,\, N_{\text{total}})$, so IDs never repeat and the data are naturally sorted by `id` within each chunk and across chunks.
- **Termination.** When all N_{total} units have been generated, `next_chunk()` returns `NULL`, signalling the end of the stream.

7.3 Application Script Structure (`streaming_bigdata.R`)

The script is organised into three sections:

Section 1 — Correctness verification (small N). For $N = 1{,}000$ and $T = 3$, run both the streaming pipeline and `plm`, and confirm that the maximum absolute differences

$$\|\hat{\beta}_{\text{online}} - \hat{\beta}_{\text{plm}}\|_\infty, \quad \|\hat{V}_0^{\text{online}} - \hat{V}_0^{\text{plm}}\|_\infty, \quad \|\hat{V}_{cr}^{\text{online}} - \hat{V}_{cr}^{\text{plm}}\|_\infty$$

are all at machine-precision level ($\lesssim 10^{-13}$).

Section 2 — Scalability benchmark. For $N \in \{10^6,\, 10^7,\, 10^8,\, 5\times 10^8\}$ (or as large as time permits), run the streaming pipeline and record:

- total wall-clock time (DGP generation + update + finalize);
- peak per-chunk memory (size of one chunk data frame in MB);
- state handle size (constant across N, reported once);
- estimation bias $\|\hat{\beta} - \beta_{\text{true}}\|_\infty$.

Section 3 — Summary table. A formatted table comparing all runs, with a note that `plm` fails beyond $N \approx 2\times 10^7$ on a 16 GB machine (from the simulation benchmark).

8 Known Limitations and Scope

- **Algorithm 3 not used.** The application module assumes a pre-specified and fixed T_{support}. If new calendar times were to appear in later chunks, the parameter dimension would need to expand (Algorithm 3), which is not implemented in the current chunk API and is left for future work.
- **New observations for existing units not handled.** The current pipeline assumes all units are new (Algorithm 1 only). Algorithm 2 (new time observation for an existing unit) is not implemented in the chunk API.
- **Sorting cost.** For very large files, the external sort in Phase 1 may require significant time and temporary disk space. Datasets originating from a database can bypass this by issuing an `ORDER BY id` query at read time.
- **Boundary scan memory.** The run-length encoding of the `id` column produces a vector of N integers ($\sim$400 MB for $N = 10^8$). For $N \geq 10^9$, this may itself become a memory concern. A future improvement would read chunks adaptively without a pre-computed boundary table.